CHAPTER 24

Drop: Weakening with a Side Effect

In the affine λ-calculus of Chapter 22, throwing a value away costs nothing. The weakening rule lets a variable sit in the context unused, and when the program finishes, the variable simply vanishes: the calculus has no files, no locks and no memory to give back.

Rust is affine too, but in Rust, throwing a value away *runs code*. When a `MutexGuard` goes out of scope, a lock is released; when a `File` does, a file descriptor is closed; when a `Vec<String>` does, every string is freed, then the vector's buffer. The typing rule is the same as in the calculus (you may discard), but the operational rule is not: discarding is an *event*, at a precise point of the program, and programs can observe it.

This chapter is about that event: when Rust runs destructors and in which order, what the compiler generates for them, and what it forbids so that no resource is released twice. Then comes the other side of the coin. Rust guarantees that a destructor runs *at most once*, not that it runs *at least once*: leaking a value is safe. That decision, taken a few weeks before Rust 1.0, is the precise reason why Rust is affine and not linear, and it still shapes how safe APIs are designed. In Lean, which has no destructors, we look at reference counting and in-place updates, and then prove, for a small resource language with a log of releases, "at most once" and, in the absence of `forget`, "exactly once".

In this chapter you will learn:

- why discarding a value in Rust is "weakening with a side effect", and what RAII guards (locks, transactions, buffered writers) do with it;
- the rules of drop order, explained line by line on a running program: locals, fields, temporaries, moves, drop flags, assignment and `mem::drop`;
- what drop glue is, what `ManuallyDrop` does, and why you can neither call `Drop::drop` yourself (E0040) nor move a field out of a type with a destructor (E0509);
- why leaking is safe in Rust: `mem::forget`, `Box::leak`, `Rc` cycles and `Weak` counts, and the history of `thread::scoped` that settled the question;
- how to design APIs that stay sound when a destructor does not run (`Vec::drain`, `thread::scope`), and what linear types would add, with `#[must_use]` as a lint-level approximation;
- how Lean manages memory without destructors, and how reference counts let a pure language update values in place;

- how to model drops in Lean and prove that no resource is released twice, that without `forget` every resource is released exactly once, and that `forget` breaks the second property.

24.1 Discarding is not free

Recall the shape of weakening. In the logic of Chapter 6 it says that an unused assumption does no harm: if $\Gamma \vdash B$ then $\Gamma, A \vdash B$. In the affine λ-calculus of Chapter 22 it says that a variable may stay unused. To talk about the moment of discarding, it helps to make it explicit with a construct $\texttt{drop}\ x; e$ that throws x away and continues with e. Its typing rule is weakening in disguise:

$$\frac{\Gamma \vdash e : \rho}{\Gamma, x : \tau \vdash \texttt{drop}\ x; e : \rho}\ \text{T-Drop}$$

and in the pure calculus its computation rule does nothing at all:

$$\texttt{drop}\ x; e \to e$$

Rust keeps the typing rule: any value may be discarded, whatever its type (a choice with consequences, as we will see). What Rust changes is the computation rule. A Rust program runs in a world with resources: memory, file descriptors, locks, connections. Writing σ for the state of that world, the rule becomes

$$\langle \sigma, \texttt{drop}\ x; e \rangle \to \langle \mathit{release}_\tau(\sigma, x), e \rangle$$

where $\mathit{release}_\tau$ is the *destructor* of the type τ: the code that gives back whatever a value of type τ holds. Weakening is still allowed, but it has become an effect. That is the title of this chapter.

RAII

The programming pattern built on this rule has a name inherited from C++: **RAII**, "resource acquisition is initialization". The idea is simpler than the name. Tie each resource to a value: acquiring the resource creates the value, and the value's destructor releases the resource. The compiler then releases it on every path out of the scope: the normal end, an early `return`, a `?`, a `break`, even a panic.

The standard library is full of such values, usually called *guards*. Here is the most common one, a `MutexGuard`, from this chapter's example `code/rust/examples/ch24_raii.rs`:

```rust
fn lock_guard() {
    let balance = Mutex::new(100_u32);
    {
        let mut guard = balance.lock().unwrap(); // acquire
        *guard -= 30;
        println!("  inside: locked by us? {}", balance.try_lock().is_err());
    } // the MutexGuard is dropped here: release
    println!("  after the block: free? {}", balance.try_lock().is_ok());
    println!("  balance = {}", balance.lock().unwrap());
}
```

```
lock guard:
  inside: locked by us? true
  after the block: free? true
  balance = 70
```

Inside the block the mutex is locked, so `try_lock` fails (it returns an error instead of waiting, which is why this single-threaded program does not deadlock). At the closing brace the guard is discarded, its destructor unlocks the mutex, and `try_lock` succeeds. There is no `unlock` method to forget to call: the only way to unlock is to stop holding the guard.

You can write your own guards by implementing the `Drop` trait, whose single method `fn drop(&mut self)` is the destructor. A good example is a database transaction that rolls back unless it is explicitly committed:

```
/// A transaction on an in-memory table. Rows are staged in `pending` and
/// reach the table only on `commit`; a transaction that is dropped without
/// being committed rolls back.
#[must_use = "a transaction rolls back unless it is committed"]
struct Transaction<'db> {
    table: &'db mut Vec<String>,
    pending: Vec<String>,
    committed: bool,
}

impl<'db> Transaction<'db> {
    fn begin(table: &'db mut Vec<String>) -> Self {
        Transaction {
            table,
            pending: Vec::new(),
            committed: false,
        }
    }

    fn insert(&mut self, row: &str) {
        self.pending.push(row.to_string());
    }

    /// Consumes the transaction: after `commit` it cannot be used again.
    fn commit(mut self) {
        self.table.append(&mut self.pending);
        self.committed = true;
    } // `self` is dropped here, and `drop` sees `committed == true`
}

impl Drop for Transaction<'_> {
    fn drop(&mut self) {
        if !self.committed {
            println!(
                "  rollback: {} pending row(s) discarded",
                self.pending.len()
            );
        }
    }
}

fn transactions() {
    let mut orders = Vec::new();

    let mut tx = Transaction::begin(&mut orders);
    tx.insert("order 1: 3 x sensor");
    tx.insert("order 2: 1 x cable");
    tx.commit();

    let mut tx = Transaction::begin(&mut orders);
    tx.insert("order 3: 9 x probe");
    // An early return, a `?` or a panic here would skip the commit:
    // the destructor rolls back either way.
    drop(tx);

    println!("  table = {orders:?}");
}
```

```
transactions:
  rollback: 1 pending row(s) discarded
  table = ["order 1: 3 x sensor", "order 2: 1 x cable"]
```

The design has two exits. The *explicit* one is `commit`, which takes `self` by value: after `tx.commit()`, `tx` has been moved and cannot be used again (Chapter 22's affine typing at work). The *implicit* one is the destructor, which rolls back every transaction discarded without a commit. The first transaction is committed, so its destructor stays silent; the second is dropped and rolls back. As the comment in the code says, the rollback also happens if anything between `begin` and `commit` returns early or panics.

Notice two things the destructor *cannot* do. It cannot take arguments, so it cannot ask the caller whether to commit. And it cannot return a value, so it cannot report failure. Both limits are real, and the standard library's `BufWriter` shows the second one:

```
fn buffered_writer() {
    let mut file_contents: Vec<u8> = Vec::new();
    {
        let mut writer = BufWriter::new(&mut file_contents);
        writer.write_all(b"t,celsius\n0,21.5\n").unwrap();
        println!(
            "  buffered, not yet written: {} bytes",
            writer.buffer().len()
        );
    } // BufWriter's destructor flushes the buffer (and ignores any error)
    println!("  after the drop, written: {} bytes", file_contents.len());
}
```

```
buffered writer:
  buffered, not yet written: 17 bytes
  after the drop, written: 17 bytes
```

A `BufWriter` collects small writes in a buffer. While it is alive, the 17 bytes sit in the buffer; when it is dropped, its destructor flushes them. If flushing failed (a full disk, a closed socket), the destructor could not tell anyone, so it ignores the error. That is why `BufWriter` also has an explicit `flush` method that returns a `Result`, and why careful code calls it: the destructor is a safety net, not the main road.

The slogan

Put the typing rule and the computation rule side by side and you get a slogan that is worth remembering, with one caveat that the rest of the chapter explains:

> Rust is **affine in its typing and linear in its dynamics**. The type checker lets you discard any value, but the compiler inserts the discarding for you, so every value that is not moved away is released when its owner goes out of scope.

The caveat is in the words "every value". As we will see, safe Rust has ways to make a value disappear *without* its destructor running, and the language explicitly allows them.

24.2 The rules of drop order

Because destructors are observable, the *order* in which they run is part of the meaning of a program. In the pure calculus, the context was a multiset (or a list up to the exchange rule) and the order in which unused variables vanished did not matter. In Rust it matters, and the language fixes it. The example `code/rust/examples/ch24_drop_order.rs` makes the rules visible with a type whose destructor prints its name:

```
/// A value that announces its own destruction.
struct Noisy(&'static str);

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("  drop {}", self.0);
    }
}
```

Each function below exercises one rule; `main` calls them in order, printing a header before each. We will go through the output line by line.

Locals: reverse declaration order

```
fn locals() {
    let _a = Noisy("a");
    let _b = Noisy("b");
    let _c = Noisy("c");
    println!("  end of scope");
} // _c, _b, _a are dropped here, in this order
```

```
locals:
  end of scope
  drop c
  drop b
  drop a
```

The three variables are dropped at the closing brace, after the last statement of the function, and in the *reverse* of the order in which they were declared: `c`, `b`, `a`. The reason is that later variables may depend on earlier ones. If `b` were a guard borrowed from `a` (as `guard` was borrowed from `balance` above), dropping `a` first would leave `b` pointing at a dead value. Dropping in reverse order is a stack discipline: the last thing set up is the first thing torn down. (The leading underscore in `_a` only silences the "unused variable" warning. It does not change when the value is dropped. A bare `_` does, as we will see in a moment.)

Fields: declaration order, after the outer destructor

```
/// A struct with its own destructor *and* fields that have destructors.
struct Pair {
    first: Noisy,
    second: Noisy,
}

impl Drop for Pair {
    fn drop(&mut self) {
        // Runs first, while the fields are still intact.
        println!("  drop Pair({}, {})", self.first.0, self.second.0);
    }
```

```
}

fn fields() {
    let _pair = Pair {
        first: Noisy("first"),
        second: Noisy("second"),
    };
    let _tuple = (Noisy("t.0"), Noisy("t.1"));
    println!("  end of scope");
}
```

```
fields:
  end of scope
  drop t.0
  drop t.1
  drop Pair(first, second)
  drop first
  drop second
```

Two locals again, so `_tuple` goes first and `_pair` second (reverse declaration order). Inside each of them, the order is different:

- The tuple's elements are dropped *in order*: `t.0`, then `t.1`. The same holds for the fields of a struct, the elements of an array and the elements of a `Vec`.
- For `Pair`, which has its own destructor, the outer destructor runs *first*, while the fields are still intact (it prints both names), and only then are the fields dropped, in declaration order: `first`, then `second`.

So a compound value is destroyed from the outside in. This is what makes it safe for `Drop::drop` to use its fields: they are still alive when it runs.

Temporaries, and `let _`

A *temporary* is a value that the program creates without giving it a name, such as the `Noisy("temporary")` passed by reference below. Temporaries are dropped at the end of the *statement* that created them:

```
fn name_length(n: &Noisy) -> usize {
    n.0.len()
}

fn temporaries() {
    // The temporary lives until the end of the statement that created it.
    let len = name_length(&Noisy("temporary"));
    println!("  after the call, len = {len}");
    // `_` is not a variable: nothing owns the value, so it is dropped at once.
    let _ = Noisy("underscore");
    println!("  after `let _ =`");
    // `_kept` is a variable: the value lives until the end of the scope.
    let _kept = Noisy("kept");
    println!("  end of scope");
}
```

```
temporaries:
  drop temporary
  after the call, len = 9
  drop underscore
  after `let _ =`
```

```
  end of scope
  drop kept
```

The first line of output is `drop temporary`: the temporary lived just long enough for `name_length` to read it, and died at the semicolon. The next lines show a classic trap. In `let _ = Noisy("underscore")`, the pattern `_` is not a variable: it binds nothing, so nothing owns the value after the statement, and it is dropped at the semicolon. `_kept`, by contrast, is a variable like any other. If you write `let _ = mutex.lock().unwrap();` to hold a lock for the rest of a function, you have in fact released it immediately.

> **NOTE**
>
> The precise rules for temporaries have more cases than we can cover here (for example, `let r = &Noisy("x");` *extends* the temporary's life to the end of the block, because otherwise `r` would dangle), and the 2024 edition, which this book uses, changed two of them: temporaries in the tail expression of a block, and in the scrutinee of an `if let` (the expression after the `=`), are now dropped earlier than in previous editions. The Rust Reference's chapter on destructors is the authority.

Moves cancel drops, and drop flags

A value is dropped by its *owner*. When a value is moved, ownership moves with it, and so does the responsibility for dropping it:

```rust
/// Takes ownership: its parameter is dropped when `consume` returns.
fn consume(n: Noisy) {
    println!("  consume({})", n.0);
}

fn moves(send: bool) {
    let x = Noisy("x");
    let _y = Noisy("y");
    if send {
        consume(x); // x is moved: this scope must no longer drop it
    }
    println!("  end of scope (send = {send})");
} // _y is dropped; x is dropped only if it was not moved (a drop flag)
```

```
moves (send = true):
  consume(x)
  drop x
  end of scope (send = true)
  drop y
moves (send = false):
  end of scope (send = false)
  drop y
  drop x
```

With `send = true`, `x` is moved into `consume`, whose parameter `n` now owns the value and drops it when `consume` returns: `drop x` appears *before* the end of `moves`, and at the end only `_y` is dropped. Dropping `x` there too would release the same value twice, a classic double free. With `send = false`, nothing is moved, and both values are dropped at the end, in reverse order.

It is the *same* function: whether `x` must be dropped at the closing brace depends on a value known only at run time. The compiler handles this with a **drop flag**, a hidden boolean in the stack frame that records whether `x` still owns its value. The move clears it, and the drop at the end of the scope is guarded by it. When the compiler can tell statically that a variable is always moved, or never, it does without the flag. Chapter 29 shows where these drops live in the compiler's intermediate representation.

Assignment and `mem::drop`

```
fn early() {
    let mut slot = Noisy("old");
    println!("  slot holds {}", slot.0);
    slot = Noisy("new"); // the old value is dropped by the assignment
    println!("  slot holds {}", slot.0);
    let guard = Noisy("guard");
    drop(guard); // std::mem::drop: release now, not at the end of the scope
    println!("  end of scope");
}
```

```
early:
  slot holds old
  drop old
  slot holds new
  drop guard
  end of scope
  drop new
```

Assigning a new value to a variable that owns an old one drops the old value *at the assignment*: `drop old` appears between the two `slot holds` lines. (Assigning to a variable whose value has been moved out drops nothing; the drop flag again.) And `drop(guard)` releases `guard` immediately, so `drop guard` appears before `end of scope`, and only `new` is left for the closing brace.

`drop` here is `std::mem::drop`, which is in the prelude. It is worth looking at its definition, because it is the whole story of this chapter in one line:

```
pub fn drop<T>(_x: T) {}
```

The body is empty. `drop` takes ownership of its argument and does nothing with it, so the argument is dropped when the function returns. `mem::drop` is not a special operation: it is weakening, packaged as a function. Since it takes its argument by value, a later use is a use after move (`code/rust/compile_fail/ch24_use_after_drop.rs`):

```
pub fn close_then_use(conn: Connection) -> usize {
    drop(conn);
    conn.host.len()
}
```

```
error[E0382]: borrow of moved value: `conn`
  --> compile_fail/ch24_use_after_drop.rs:12:5
   |
10 | pub fn close_then_use(conn: Connection) -> usize {
   |                       ---- move occurs because `conn` has type `Connection`, which does
not implement the `Copy` trait
11 |     drop(conn);
   |          ---- value moved here
```

```
12 |     conn.host.len()
   |     ^^^^^^^^^ value borrowed here after move
   |
note: if `Connection` implemented `Clone`, you could clone the value
  --> compile_fail/ch24_use_after_drop.rs:5:1
   |
 5 | pub struct Connection {
   | ^^^^^^^^^^^^^^^^^^^^^ consider implementing `Clone` for this type
...
11 |     drop(conn);
   |          ---- you could clone this value
```

Read the diagnostic with Chapter 22 in mind: `conn` is not `Copy`, it was consumed by the call `drop(conn)` ("value moved here", as for any other call), and the context no longer contains it.

Order breaks exchange, operationally

In Chapter 6 we saw that Rust keeps the exchange rule: the order of the assumptions in a context does not matter for typing. The `locals` example shows that it matters for the *behavior*: swap the declarations of `_a` and `_b` and the output changes. So exchange holds at the level of the typing judgment, which is where we stated it, but not at the level of observable behavior. This is a small but instructive example of why the claim "Rust is based on logic X" always needs to say *at which level*.

24.3 Drop glue, and two things you cannot do

Most types do not implement `Drop` themselves. A `(u64, String)` has no `drop` method, yet dropping it frees the string. The compiler generates, for every type, a function called the type's **drop glue**: first call `Drop::drop` if the type implements it, then run the drop glue of each field, in declaration order. That is exactly the "outside in" order we observed for `Pair`. A type whose drop glue does nothing at all is one that needs no dropping, and `std::mem::needs_drop::<T>()` tells you which is which:

```
fn drop_glue() {
    println!("  u64                 {}", mem::needs_drop::<u64>());
    println!("  &String             {}", mem::needs_drop::<&String>());
    println!("  String              {}", mem::needs_drop::<String>());
    println!(
        "  (u64, String)       {}",
        mem::needs_drop::<(u64, String)>()
    );
    println!(
        "  Option<Box<u8>>     {}",
        mem::needs_drop::<Option<Box<u8>>>()
    );
    println!("  Noisy               {}", mem::needs_drop::<Noisy>());
    println!(
        "  ManuallyDrop<String> {}",
        mem::needs_drop::<ManuallyDrop<String>>()
    );
}
```

```
needs_drop (does the type have drop glue?):
  u64                 false
  &String             false
  String              true
  (u64, String)       true
  Option<Box<u8>>     true
  Noisy               true
  ManuallyDrop<String> false
```

A `u64` owns nothing. A `&String` is a borrow: it owns nothing either, and dropping the reference leaves the string alone (borrowing is the subject of Part VI). A `String` owns a heap buffer. A tuple needs dropping if any of its fields does, and so does an `Option<Box<u8>>`, whose `Some` case owns a box. `Noisy` has a `Drop` impl. The last line we explain below. (The documentation allows `needs_drop` to answer `true` conservatively for a type that does not really need it; it never answers `false` for one that does.)

Chapter 23 explained why a type cannot be both `Copy` and `Drop` (E0184): a bitwise copy of a value with a destructor would run that destructor twice. The same concern, "never release a resource twice", explains two more rules.

You cannot call `Drop::drop`

```
pub fn release_early(lock: Lock) {
    lock.drop();
}
```

```
error[E0040]: explicit use of destructor method
  --> compile_fail/ch24_explicit_destructor.rs:15:10
   |
15 |     lock.drop();
   |          ^^^^ explicit destructor calls not allowed
   |
help: consider using `drop` function
   |
15 -     lock.drop();
15 +     drop(lock);
   |
```

The method `Drop::drop` takes `&mut self`, not `self`: after the call, `lock` would still be a valid variable that owns its value, and the compiler would drop it again at the end of the scope, running the destructor a second time. So the compiler forbids calling the method directly (E0040), and suggests the function `drop(lock)`, which moves the value and so cancels the automatic drop. The destructor is something that *happens to* a value, at most once; it is not something you call.

You cannot move a field out of a `Drop` type

```
pub fn steal_rows(tx: Transaction) -> Vec<String> {
    tx.pending
}
```

```
error[E0509]: cannot move out of type `Transaction`, which implements the `Drop` trait
  --> compile_fail/ch24_move_out_of_drop.rs:15:5
   |
15 |     tx.pending
   |     ^^^^^^^^^^
   |     |
   |     cannot move out of here
   |     move occurs because `tx.pending` has type `Vec<String>`, which does not implement
the `Copy` trait
   |
help: consider cloning the value if the performance cost is acceptable
   |
15 |     tx.pending.clone()
   |               ++++++++
```

For an ordinary struct, moving one field out is a *partial move* (Chapter 22): the struct becomes partially uninitialized, and at the end of the scope the compiler drops only the fields that are still there. For a struct with a destructor this is impossible, because `Drop::drop` receives `&mut self` and may read every field: it needs the whole value. So the compiler forbids it (E0509). The usual workarounds leave a valid value behind: `std::mem::take` swaps the field with its default (an empty `Vec`), `Option::take` swaps an `Option` field with `None`, and `std::mem::replace` swaps in a value you choose. The `Transaction` above sidesteps the problem in `commit`: it moves the rows with `self.table.append(&mut self.pending)`, which empties `pending` through a mutable reference instead of moving it out.

Destructors and panics

When a thread panics, Rust (by default) *unwinds* the stack: it leaves every active function, and on the way out it drops every live local, exactly as a `return` would.

```
fn unwinding() {
    // Replace the default panic message (printed on stderr) with nothing.
    std::panic::set_hook(Box::new(|_| {}));
    let result = std::panic::catch_unwind(|| {
        let _probe = Noisy("probe guard");
        panic!("sensor disconnected");
    });
    let _ = std::panic::take_hook(); // restore the default hook
    println!("  caught a panic: {}", result.is_err());
}
```

```
unwinding:
  drop probe guard
  caught a panic: true
```

`catch_unwind` runs the closure and catches the panic; on its way out of the closure, the panic dropped `_probe`. This is what makes guards reliable: a `MutexGuard` dropped during a panic releases its lock (and marks the mutex "poisoned", so that the next user knows the data may be half-updated). Two exceptions: a program built with `panic = "abort"` does not unwind, so no destructors run; and a destructor that panics *during* unwinding aborts the process, which is why destructors should not panic.

`ManuallyDrop`

Sometimes you want to switch the automatic drop off. `std::mem::ManuallyDrop<T>` is a wrapper whose drop glue does nothing (hence `false` in the table above):

```
fn manually_drop() {
    let wrapped = ManuallyDrop::new(Noisy("wrapped"));
    println!("  holding {}", wrapped.0);
    let unwrapped = ManuallyDrop::into_inner(ManuallyDrop::new(Noisy("unwrapped")));
    println!("  holding {}", unwrapped.0);
    println!("  end of scope");
} // `unwrapped` is dropped; `wrapped` is not (its destructor never runs)
```

```
ManuallyDrop:
  holding wrapped
  holding unwrapped
  end of scope
  drop unwrapped
```

Only `unwrapped` is dropped: `ManuallyDrop::into_inner` gave the value back, with its ordinary destructor, while `wrapped` was never unwrapped. `ManuallyDrop` is mostly used in `unsafe` code that needs exact control over when a field is destroyed (dropping it in place, `ManuallyDrop::drop`, is `unsafe`, because the compiler can no longer check that it happens at most once; Chapter 32 discusses such obligations). But creating a `ManuallyDrop` is safe: a first hint that in Rust, "the destructor will run" is not a promise.

24.4 Affine, not linear

In the lattice of Chapter 21, a *linear* logic has neither weakening nor contraction (every assumption is used exactly once), and an *affine* logic keeps weakening (at most once). For Rust this means:

Guarantee	Linear system	Rust
a value is consumed at most once	yes	yes (non-`Copy` types)
a value is consumed at least once	yes	no
its destructor runs	by construction	usually, not guaranteed
a protocol is run to completion	can be enforced	only checked at run time

This section is about the second and third lines: Rust lets you discard any value, even *without* running its destructor.

`mem::forget` and `Box::leak`

The function `std::mem::forget<T>(t: T)` takes ownership of a value and makes sure its destructor never runs. It is a safe function. So is `Box::leak`, which turns a `Box<T>` into a reference `&'static mut T` to a heap allocation that will never be freed. From `code/rust/examples/ch24_leaks_and_scopes.rs`:

```
fn forgetting() {
    let kept = Noisy("kept");
    let forgotten = Noisy("forgotten");
    mem::forget(forgotten); // safe: the destructor will never run
    let leaked: &'static mut Noisy = Box::leak(Box::new(Noisy("leaked")));
    leaked.0 = "leaked (renamed)"; // a `'static` reference to a never-freed box
    println!("  end of scope, still holding {}", kept.0);
}
```

```
mem::forget and Box::leak:
  end of scope, still holding kept
  drop kept
```

Three values were created and only one destructor ran. The leaked box lives on, and the reference to it is valid for the rest of the program (that is what `'static` means), which is occasionally useful for data created once at startup. Neither needed `unsafe`. Making `forget` safe may look strange; the surprising fact is that making it unsafe would not help, because safe Rust can leak values without it.

`Rc` cycles and `Weak` counts

An `Rc<T>` (Chapter 23) is a pointer with a reference count. `Rc::clone` copies the pointer and increments the count; dropping an `Rc` decrements it; when the count reaches zero, the value inside is

dropped. Combined with interior mutability (`RefCell` , Chapter 26), `Rc` lets you build arbitrary graphs. A tree in which children are owned by their parent, and point back to it, is the textbook example:

```
/// A node of a tree: children are owned (strong `Rc`), the parent pointer is
/// a `Weak`, which does not keep the parent alive.
struct Node {
    name: &'static str,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

impl Node {
    fn new(name: &'static str) -> Rc<Node> {
        Rc::new(Node {
            name,
            parent: RefCell::new(Weak::new()),
            children: RefCell::new(Vec::new()),
        })
    }
}

impl Drop for Node {
    fn drop(&mut self) {
        println!("  drop node {}", self.name);
    }
}

fn adopt(parent: &Rc<Node>, child: &Rc<Node>) {
    parent.children.borrow_mut().push(Rc::clone(child));
    *child.parent.borrow_mut() = Rc::downgrade(parent);
}

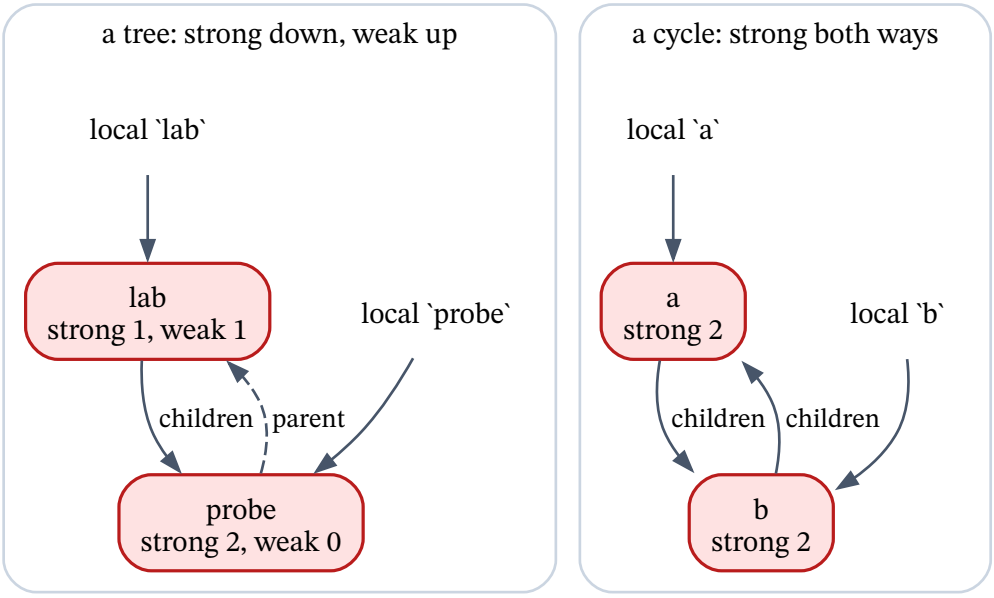
fn counts(label: &str, node: &Rc<Node>) {
    println!(
        "  {label}: strong = {}, weak = {}",
        Rc::strong_count(node),
        Rc::weak_count(node)
    );
}

fn tree() {
    let lab = Node::new("lab");
    let probe = Node::new("probe");
    adopt(&lab, &probe);
    counts("lab  ", &lab);
    counts("probe", &probe);
    let parent = probe.parent.borrow().upgrade().map(|p| p.name);
    println!("  probe's parent: {parent:?}");
    drop(lab); // the last strong reference to "lab": the whole tree goes
    let parent = probe.parent.borrow().upgrade().map(|p| p.name);
    println!("  probe's parent after drop(lab): {parent:?}");
    counts("probe", &probe);
}
```

```
a tree with Weak parent pointers:
  lab  : strong = 1, weak = 1
  probe: strong = 2, weak = 0
  probe's parent: Some("lab")
  drop node lab
  probe's parent after drop(lab): None
  probe: strong = 1, weak = 0
  drop node probe
```

`Rc<Node>` is a **strong** reference: it owns a share of the node and keeps it alive. `Weak<Node>` is a **weak** reference: it remembers where the node is but does not keep it alive, and must be *upgraded* to an `Rc` (which fails if the node is gone) before use. Read the counts with the left half of the figure below:

`lab` is held strongly by its local and weakly by the child's `parent` field; `probe` is held strongly by its local and by `lab`'s `children`. Then `drop(lab)` drops the only strong reference to `lab`: its destructor runs, the drop glue of its `children` vector releases one strong reference to `probe`, and the child's weak pointer no longer upgrades (`None`).



Two graphs of reference-counted nodes. Left: a tree with strong pointers from parent to child and a weak pointer back; the counts reach zero. Right: two nodes pointing strongly at each other; when the locals go away, each count stays at one and neither node is ever freed.

Now make *both* directions strong:

```
fn cycle() {
    let a = Node::new("a");
    let b = Node::new("b");
    a.children.borrow_mut().push(Rc::clone(&b)); // a -> b (strong)
    b.children.borrow_mut().push(Rc::clone(&a)); // b -> a (strong): a cycle
    counts("a", &a);
    counts("b", &b);
    println!("  end of scope");
} // a and b lose one strong reference each, but neither count reaches 0
```

```
a cycle of strong pointers:
  a: strong = 2, weak = 0
  b: strong = 2, weak = 0
  end of scope
```

Each node has a strong count of 2: its local and the other node's `children`. At the end of `cycle` the locals are dropped, each count goes down to 1, and there it stays: the program's complete output, which ends with `end of main`, never contains `drop node a` or `drop node b`. The two nodes keep each other alive, unreachable, until the process exits. Reference counting, unlike a tracing garbage collector, cannot see that a group of objects is unreachable from outside. The standard fix is the one used in the tree: make the back edges `Weak`.

The consequence for the type theory is direct. Any safe code can build such a cycle, so any safe code can put a value of *any* type `T` into one and lose it forever, without its destructor ever running. In other words, `forget` can be written in safe Rust (with an `Rc` cycle containing the value), and marking `mem::forget` as `unsafe` would only have been a polite fiction.

The Leakpocalypse

This is the moment the language committed to its position. In the spring of 2015, shortly before Rust 1.0, the standard library had a function `std::thread::scoped`. It spawned a thread allowed to *borrow* data from the parent's stack frame, and returned a guard, `JoinGuard`, whose destructor waited for (joined) the thread. The safety argument: the guard cannot outlive the borrowed data (the borrow checker ensures that), and dropping the guard joins the thread; so the thread finishes before the borrowed data goes away.

The argument has a hidden premise: *the guard's destructor runs*. Someone noticed that it need not. Put the guard into an `Rc` cycle and it is never dropped; the parent function returns, its stack frame is reused, and the child thread keeps reading and writing memory that now belongs to someone else: a use-after-free in a program with no `unsafe` in it.

The episode, nicknamed the *Leakpocalypse* in the community, was resolved in a few weeks, and the resolution has three parts that still hold:

1. `thread::scoped` was taken out of the stable standard library before Rust 1.0 (and later removed altogether).
2. `mem::forget`, which until then had been an `unsafe` function, was made safe, recording officially that leaking a value is not a memory-safety violation.
3. A rule for everyone who writes `unsafe` code: **soundness must not depend on a destructor running.** A destructor may be used for cleanup and for the comfortable path; it must never be the only thing that stops undefined behavior.

Leaks are still bugs, but bugs of the same class as infinite loops and deadlocks, which Rust's type system does not prevent either (Chapter 9 discussed what "going wrong" means), not of the class of use-after-free, which it does.

Designing for leaks: `Vec::drain` and `thread::scope`

How do you build an API whose safety needs some cleanup to happen, under this rule? There are two main techniques, and the standard library shows both.

The first is to make the half-finished state *safe*, if not useful, before handing out the guard. `Vec::drain(range)` returns an iterator that removes a range of elements; its destructor shifts the tail down to close the gap. While the iterator is alive, the vector has a hole in the middle, and if the iterator were forgotten with the vector left as it was, the vector would later drop the elements in the hole a second time. So `drain` shrinks the vector's length *first*:

```
fn forgotten_drain() {
    let mut readings = vec![10, 11, 12, 13, 14, 15];
    let drain = readings.drain(1..3); // would remove 11 and 12
```

```
    mem::forget(drain); // its destructor, which repairs the vector, never runs
    println!("  after forgetting the drain: {readings:?}");
}
```

```
a forgotten Vec::drain:
  after forgetting the drain: [10]
```

After forgetting the drain, the vector reports only the element before the range; the other five are leaked. The documentation warns that a forgotten `Drain` may lose elements (exactly which is an implementation detail), but the result is *safe*: nothing is dropped twice and nothing dangling is read. The worst case was turned from undefined behavior into lost data.

The second technique is to not hand out a guard at all, and keep control instead. The modern replacement for `thread::scoped` is `std::thread::scope`, stable since Rust 1.63:

```
fn scoped_threads() {
    let samples = vec![3, 1, 4, 1, 5, 9, 2, 6];
    let (left, right) = samples.split_at(samples.len() / 2);
    // Both threads borrow `samples` from this stack frame. `thread::scope`
    // joins them before it returns, so the borrow cannot outlive the data.
    let (sum_left, sum_right) = thread::scope(|s| {
        let h1 = s.spawn(|| left.iter().sum::<i32>());
        let h2 = s.spawn(|| right.iter().sum::<i32>());
        (h1.join().unwrap(), h2.join().unwrap())
    });
    println!(
        "  sums: {sum_left} + {sum_right} = {}",
        sum_left + sum_right
    );
    println!("  samples still owned here: {samples:?}");
}
```

```
thread::scope:
  sums: 9 + 22 = 31
  samples still owned here: [3, 1, 4, 1, 5, 9, 2, 6]
```

`thread::scope` passes its closure a scope handle `s`. Threads spawned with `s.spawn` may borrow anything that outlives the scope, here the two halves of `samples`. The crucial difference from the old design is *who* waits for the threads: not a destructor, but `thread::scope` itself, which joins every thread after the closure returns, and only then returns. A caller cannot forget a guard it never receives. The guarantee comes from control flow instead of a destructor, and this closure-passing style is the standard way to get "exactly once" behavior in an affine language; Chapter 25 uses it again.

24.5 What linearity would buy

Suppose Rust had genuinely linear types, whose values *must* be consumed by one of a designated set of operations (no weakening, in the terms of Chapter 21). What could we express that we cannot today?

- **Mandatory decisions.** A `Transaction` that must be either committed or rolled back explicitly, so that forgetting to decide is a compile-time error rather than a silent rollback.
- **Fallible cleanup.** A file that must be closed by a `close(self) -> Result<(), Error>` call, so that the error from the final flush cannot be ignored, as `BufWriter`'s destructor ignores it today.

- **Protocol completion.** A network session or a lab instrument (Chapter 25) whose protocol must be run to its final state, not abandoned half-way.

In each case the consuming operation, being an ordinary method that the programmer must call, can take arguments and return results, which a destructor cannot.

`#[must_use]`: a lint, not a rule

Rust has one feature that points in this direction. The attribute `#[must_use]` on a type (or a function) makes the compiler *warn* when a value of that type is produced and immediately thrown away. Our `Transaction` carries one:

```
#[must_use = "a transaction rolls back unless it is committed"]
struct Transaction<'db> { /* ... */ }
```

If you write the statement `Transaction::begin(&mut orders);` on its own, the compiler warns "unused `Transaction` that must be used", repeats the note from the attribute, and suggests "use `let _ = ...` to ignore the resulting value". That suggestion shows how weak the guarantee is. `#[must_use]` checks only that the value of an *expression statement* is not discarded on the spot; `let _ =`, an unused variable or a call to `drop` all satisfy it. It is a lint: not part of type checking, possible to turn off, and nothing `unsafe` code may rely on. It approximates "must be used" the way a unit test approximates a proof.

Why Rust is not linear, and the state of the discussion

Adding linear types to Rust after the fact is hard. Every generic function may drop a value of its type parameter (`fn store<T>(x: T) {}` compiles for every `T`), and the standard library is full of such functions; unwinding drops every live value; `mem::forget` and `Rc` cycles are safe. Linear types would need a new kind of bound meaning "may not be dropped implicitly" (or "may not be leaked"), and every generic API would have to be revisited to decide whether it supports such types.

Proposals along these lines have been discussed in the Rust community for years, under names such as "linear types", "must-move types" and a `Leak` trait. None is part of the language this book uses: the standard library of Rust 1.99 has no such marker trait. If the topic interests you, check the current state of the discussion. Other languages have taken the step: GHC Haskell, for example, has an extension that adds linear function types.

24.6 Lean: memory without destructors

Lean has no `Drop`: a Lean programmer never writes code that runs when a value dies. Memory is still managed precisely and deterministically, by **reference counting**: every heap object carries a count of the references to it and is freed when the count reaches zero, at that moment. This is Rust's `Rc`, applied by the runtime to every object. Values built by pure code cannot form cycles (an immutable value cannot point to something created after it), so for pure data reference counting frees everything; mutable references such as `IO.Ref` can be made to form cycles, and those are not freed, as in Rust.

Other resources are handled by the runtime too. The documentation of Lean's file handles, `IO.FS.Handle`, says that there is no explicit operation to close a file: when the last reference to a handle

is dropped, the file is closed automatically. So Lean has RAII for the resources its runtime provides, implemented in C, but a Lean program cannot attach its own code to the death of a value.

Functional, but in place

Chapter 2 previewed the most interesting consequence of reference counting. In pure Lean, `xs.set! 0 99` does not modify `xs`; it returns a new array that differs from `xs` in position 0. Done naively, that copies the whole array on every update. But the runtime knows the reference count of `xs`. If it is 1, nobody else can see `xs`, and nobody will notice if the update is done *in place*. Only a shared array has to be copied. This is often summed up as "functional but in place".

The function `dbgTraceIfShared msg a` lets us watch the runtime decide. It returns `a` unchanged, and as a side effect, if `a` is shared, prints the message `msg` prefixed by `shared RC`. The Lean file `code/lean/Book/Ch24/InPlace.lean` uses it on a function that overwrites the first element of an array:

```
/-- Overwrite the first element. `Array.set!` updates the array in place when
it is the only reference to it, and copies it otherwise; `dbgTraceIfShared`
reports the copying case. -/
def bump (xs : Array Nat) : Array Nat :=
  (dbgTraceIfShared "bump" xs).set! 0 99

/-- A fresh array whose contents depend on the argument `n`, so that Lean
cannot precompute it once and share it. -/
def fresh (n : Nat) : Array Nat := Array.replicate n 0
```

The reason for `fresh` is subtle and comes back below. First, an array with a single reference:

```
/-- The array from `fresh n` has exactly one reference, which `bump` consumes. -/
def bumpFresh (n : Nat) : Array Nat := bump (fresh n)

/-- info: #[99, 0, 0] -/
#guard_msgs in
#eval bumpFresh 3
```

The only output is the result: no `shared RC` message, so the array built by `fresh 3` was updated in place. Now keep a second reference to the array:

```
/-- Here `xs` is still needed after the call: it is shared, so `bump` copies. -/
def bumpAndKeep (n : Nat) : Array Nat × Array Nat :=
  let xs := fresh n
  (bump xs, xs)

/--
info: shared RC bump
---
info: (#[99, 0, 0], #[0, 0, 0])
-/
#guard_msgs in
#eval bumpAndKeep 3
```

Here `#guard_msgs` checks two messages, separated by `---`. The first, `shared RC bump`, is printed by the runtime while `bump` runs: `xs` is still needed for the second component of the pair, so its count is 2, and `set!` copies. The second, the result, shows why the copy was necessary: the pair holds the updated array *and* the original one, unchanged, as an immutable language demands.

The same reasoning applies to loops. A `for` loop in `Id.run do` that updates a `let mut` array is, underneath, a chain of functions each of which consumes the array and returns the next version. If no one else holds a reference, every update is in place:

```
/-- A loop that writes every slot. The array is never shared, so all `n`
updates happen in place: no message is printed. -/
def stamp (n : Nat) : Array Nat := Id.run do
  let mut xs := fresh n
  for i in [0:n] do
    xs := (dbgTraceIfShared "stamp" xs).set! i (i + 1)
  return xs

/-- info: #[1, 2, 3, 4, 5] -/
#guard_msgs in
#eval stamp 5

/-- The same loop, but every intermediate version is kept in a history. Each
update now finds the array shared (the history holds a reference), and copies. -/
def stampWithHistory (n : Nat) : List (Array Nat) := Id.run do
  let mut xs := fresh n
  let mut history := []
  for i in [0:n] do
    history := xs :: history
    xs := (dbgTraceIfShared "history" xs).set! i (i + 1)
  return (xs :: history).reverse

/--
info: shared RC history
shared RC history
shared RC history
---
info: [#[0, 0, 0], #[1, 0, 0], #[1, 2, 0], #[1, 2, 3]]
-/
#guard_msgs in
#eval stampWithHistory 3
```

`stamp 5` prints only its result: five updates, no copies. `stampWithHistory 3` keeps every intermediate version in a list. Now each update finds the array shared (the list holds a reference to it), and the runtime reports three copies, one per iteration. The result shows that the copies were needed: the history really contains four different arrays.

Finally, the reason for `fresh`:

```
/-- A closed term: computed once, stored, and shared by every use. -/
def calibration : Array Nat := #[5, 6, 7]

/--
info: shared RC bump
---
info: #[99, 6, 7]
-/
#guard_msgs in
#eval bump calibration
```

`calibration` is a *closed term*: a value that does not depend on any argument. Lean computes such values once and stores them, like a Rust `static`, so every use of `calibration` refers to the same stored array, which is therefore shared, and `bump` must copy it. That is why the earlier examples build their arrays inside a function, from an argument.

Compare the two languages. In Rust, the uniqueness of a `Vec` you own, or of a `&mut` borrow, is established *statically*, so in-place mutation is simply what the code says. In Lean, uniqueness is checked

dynamically, and the outcome affects only performance, never the result. Rust has the dynamic version too: `Rc::make_mut` returns a mutable reference to the value inside an `Rc`, cloning it first if the count is above one. That is Lean's strategy as a library function.

What the logic sees

The Lean counterpart of "the typing rule does not see the destructor" is this: to Lean's logic, `dbgTraceIfShared` is the identity function. Its definition in the core library is `a`; the message comes from a native implementation that the compiler substitutes at run time. So proofs cannot observe sharing:

```
/-- To the logic, `dbgTraceIfShared` is the identity function: the message is
a side channel that proofs cannot observe. -/
theorem dbgTraceIfShared_eq {α : Type} (msg : String) (a : α) :
    dbgTraceIfShared msg a = a := rfl

/-- Whether `bump` copies or not, its result is the same value. -/
theorem bump_size (xs : Array Nat) : (bump xs).size = xs.size := by
  simp [bump, dbgTraceIfShared_eq]

theorem bump_first (xs : Array Nat) (h : 0 < xs.size) :
    (bump xs)[0]? = some 99 := by
  simp [bump, dbgTraceIfShared_eq, h]

theorem bump_rest (xs : Array Nat) (i : Nat) (hi : 0 < i) :
    (bump xs)[i]? = xs[i]? := by
  simp [bump, dbgTraceIfShared_eq, Nat.ne_of_lt hi]

/-- The two results of `bumpAndKeep` are different values, as the semantics
of an immutable language demands: the copy is what makes this true. -/
theorem bumpAndKeep_spec (n : Nat) (h : 0 < n) :
    (bumpAndKeep n).1[0]? = some 99 ∧ (bumpAndKeep n).2[0]? = some 0 := by
  simp [bumpAndKeep, bump, fresh, dbgTraceIfShared_eq, h]
```

`dbgTraceIfShared_eq` is proved by `rfl`: both sides are equal by definition. With it, `simp` sees through `bump`, and the other theorems describe its result exactly: the size is unchanged, position 0 holds 99, every other position is unchanged (`Nat.ne_of_lt hi` turns `0 < i` into `0 ≠ i`, which `simp` needs), and the two components of `bumpAndKeep` differ. None of these statements mentions reference counts, and none could: the runtime's message is a side channel, as a Rust destructor is invisible to rule T-Drop.

24.7 A drop log in Lean

We now turn the slogan "affine in typing, linear in dynamics" into theorems. The plan is to define a tiny language in which variables own resources, an evaluator that records every acquisition and release in a log (the formal counterpart of the lines our `Noisy` values print), and then prove three properties of the log and one of a static checker. Everything is in `code/lean/Book/Ch24/DropLog.lean`.

The language

```
/-- Variables are names; resources are numbered in order of acquisition. -/
abbrev Var := String
abbrev Res := Nat

/-- The statements of the language. -/
inductive Stmt where
  | new (x : Var)          -- let x = Resource::new();
```

```
  | move (x y : Var)       -- let y = x;
  | drop (x : Var)         -- drop(x);
  | forget (x : Var)       -- mem::forget(x);
  | read (x : Var)         -- x.read();  (uses x without consuming it)
  deriving Repr, DecidableEq

/-- What the log records. -/
inductive Event where
  | acquire (r : Res)
  | release (r : Res)
  deriving DecidableEq
```

Each statement is a miniature Rust statement, as the comments show: `drop` and `forget` consume a variable, with and without running the destructor, and `read` uses one without consuming it. (Resources are numbered `0, 1, 2, ...`; a `Repr` instance in the file prints them as `r0, r1, ...`.)

The state of a running program is the list of declared variables, the next fresh resource number, and the log:

```
/-- A slot is a declared variable with its *drop flag*: `some r` if the
variable owns resource `r`, `none` if its value has been moved out. -/
abbrev Slot := Var × Option Res

structure State where
  slots : List Slot      -- every declared variable, newest first
  next  : Res            -- the next fresh resource
  log   : List Event     -- what has happened, oldest first

def State.init : State := ⟨[], 0, []⟩
```

The `Option Res` in a slot is exactly Rust's drop flag: `some r` means "owns `r`, drop it at the end of the scope"; `none` means "moved out". A variable stays in the list after a move, as it stays in scope in Rust. New variables go at the front, so the list is in reverse declaration order, and a variable that shadows an older one with the same name is found first, as in Rust.

The heart of the evaluator is `take`, which moves a value out of a variable:

```
/-- Take the value out of the newest slot named `x`, clearing its drop flag.
This is the run-time version of "use of moved value". -/
def take (x : Var) : List Slot → Except String (Res × List Slot)
  | [] => .error s!"cannot find value `{x}` in this scope"
  | (y, o) :: rest =>
    if y = x then
      match o with
      | some r => .ok (r, (y, none) :: rest)
      | none => .error s!"use of moved value: `{x}`"
    else
      match take x rest with
      | .ok (r, rest') => .ok (r, (y, o) :: rest')
      | .error e => .error e
```

It walks the list to the newest slot named `x`. If that slot owns a resource, it returns the resource and the list with the slot's flag cleared; if the flag is already clear, it fails with Rust's message for E0382. `Except String α` is Lean's counterpart of `Result<α, String>` (Chapter 2). Every statement except `new` uses `take`:

```
/-- One statement. Only `new` and `drop` write to the log. -/
def step (s : State) : Stmt → Except String State
  | .new x =>
```

```
    .ok ⟨(x, some s.next) :: s.slots, s.next + 1, s.log ++ [.acquire s.next]⟩
  | .move x y =>
    match take x s.slots with
    | .ok (r, slots) => .ok ⟨(y, some r) :: slots, s.next, s.log⟩
    | .error e => .error e
  | .drop x =>
    match take x s.slots with
    | .ok (r, slots) => .ok ⟨slots, s.next, s.log ++ [.release r]⟩
    | .error e => .error e
  | .forget x =>
    match take x s.slots with
    | .ok (_, slots) => .ok ⟨slots, s.next, s.log⟩
    | .error e => .error e
  | .read x =>
    match take x s.slots with
    | .ok _ => .ok s
    | .error e => .error e
```

The anonymous constructor `⟨slots, next, log⟩` builds a `State`. `new` pushes a slot and logs an acquisition. `move` gives the value to a new slot `y` and logs nothing, because a move runs no code. `drop` logs a release. `forget` takes the value and logs *nothing*: no one owns the resource any more, and no one released it. `read` only checks that `x` still owns its value. Finally, the end of the scope:

```
/-- The end of the scope: release every resource that is still owned, newest
slot first (reverse declaration order). Moved-out slots are skipped: that is
what the drop flag is for. -/
def dropAll : List Slot → List Event → List Event
  | [], log => log
  | (_, some r) :: rest, log => dropAll rest (log ++ [.release r])
  | (_, none) :: rest, log => dropAll rest log

def exec (s : State) : List Stmt → Except String State
  | [] => .ok s
  | st :: prog =>
    match step s st with
    | .ok s' => exec s' prog
    | .error e => .error e

/-- Run a program in a fresh scope and return the complete log. -/
def run (prog : List Stmt) : Except String (List Event) :=
  match exec State.init prog with
  | .ok s => .ok (dropAll s.slots s.log)
  | .error e => .error e
```

`dropAll` releases, newest first, every resource whose flag is set. `exec` runs the statements, stopping at the first error, and `run` runs a program in a fresh scope and then drops what is left:

```
/-- The `moves` function of `ch24_drop_order.rs`, with `send = true`:
`x` is moved into `consume`, which drops it; `y` is dropped at the end. -/
def movesProgram : List Stmt :=
  [.new "x", .new "y", .move "x" "n", .drop "n"]

/-- info: Except.ok [acquire r0, acquire r1, release r0, release r1] -/
#guard_msgs in
#eval run movesProgram

/-- Without the move, both are dropped at the end, newest first. -/
def localsProgram : List Stmt := [.new "a", .new "b", .new "c"]

/--
info: Except.ok [acquire r0, acquire r1, acquire r2, release r2, release r1, release r0]
-/
#guard_msgs in
#eval run localsProgram
```

```
/-- info: Except.error "use of moved value: `conn`" -/
#guard_msgs in
#eval run [.new "conn", .drop "conn", .read "conn"]

/-- info: Except.ok [acquire r0] -/
#guard_msgs in
#eval run [.new "conn", .forget "conn"]
```

Compare with the Rust output. `movesProgram` is `moves` with `send = true`: `x` (`r0`) is moved into `n` and dropped there, before the end of the scope drops `y` (`r1`), the order the Rust program printed. `localsProgram` is `locals`: three releases in reverse order. The third program fails with the message of E0382. The fourth forgets its resource: an acquisition with no release, a leak.

Counting

The properties we want are about how many times each resource appears in the log. We count three things:

```
/-- How many slots own resource `r` right now. -/
def held : List Slot → Res → Nat
  | [], _ => 0
  | (_, o) :: rest, r => (if o = some r then 1 else 0) + held rest r

/-- How many times the log says `r` was acquired, and released. -/
def acquired (log : List Event) (r : Res) : Nat := log.count (.acquire r)
def released (log : List Event) (r : Res) : Nat := log.count (.release r)
```

`held slots r` counts the variables that currently own `r`; `acquired` and `released` use the library's `List.count`. (The file also proves small `@[simp]` lemmas such as `released (log ++ [.release r']) r = released log r + (if r' = r then 1 else 0)`, which `simp` then uses automatically.) The one lemma with real content says that `take` removes exactly one owner of the taken resource and changes no other count:

```
/-- Taking a value out of the slots removes exactly one owner of `r` (if the
taken resource is `r`) and leaves every other count unchanged. -/
theorem take_held {x : Var} {slots slots' : List Slot} {r : Res}
    (h : take x slots = .ok (r, slots')) (q : Res) :
    held slots q = held slots' q + (if r = q then 1 else 0) := by
  induction slots generalizing slots' with
  | nil =>
    -- `take x []` is an error, so `h` is impossible.
    simp [take] at h
  | cons slot rest ih =>
    obtain ⟨y, o⟩ := slot
    by_cases hyx : y = x
    · -- The newest slot is `x`: it must own a resource, which is taken.
      cases o with
      | none => simp [take, hyx] at h
      | some r₀ =>
        simp only [take, hyx, ↓reduceIte, Except.ok.injEq, Prod.mk.injEq] at h
        obtain ⟨rfl, rfl⟩ := h
        simp only [held_cons, Option.some.injEq, reduceCtorEq, ↓reduceIte]
        omega
    · -- Another variable: the value is taken from the rest of the list.
      simp only [take, hyx, ↓reduceIte] at h
      split at h
      · rename_i r₁ rest' htake
        simp only [Except.ok.injEq, Prod.mk.injEq] at h
        obtain ⟨rfl, rfl⟩ := h
        have := ih htake
        simp only [held_cons]
```

```
        omega
      · simp at h
```

For every resource `q`, the old number of owners equals the new number plus one if `q` is the taken resource. The proof is by induction on the list of slots, following the definition of `take`:

- `induction slots generalizing slots'` makes the induction hypothesis `ih` hold for *every* `slots'`, which we need because the recursive call produces a different list.
- In the `nil` case, `take x []` is an error, so `h : take x [] = .ok ...` is false: `simp [take] at h` computes the left side and closes the goal.
- In the `cons` case, `obtain ⟨y, o⟩ := slot` splits the first slot into its name and flag, and `by_cases hyx : y = x` asks whether it is the slot we want.
- If it is, `cases o` splits on the flag. With `none`, `take` fails and `h` is again impossible. With `some r₀`, `simp only [...] at h` computes `take` and turns `h` into `r₀ = r` and `(y, none) :: rest = slots'`, which `obtain ⟨rfl, rfl⟩ := h` substitutes everywhere. `simp only` then rewrites `some r₀ = some q` to `r₀ = q` and the cleared slot's contribution to zero, and `omega` finishes. (`omega` decides linear arithmetic over `Nat` and `Int`, and treats any other term, such as an `if`, as an unknown number.)
- If it is not, `take` recurses. `split at h` considers the two results of the recursive call; in the successful one, `rename_i` names the pieces, `ih htake` gives the equation for `rest`, and `omega` adds the first slot's unchanged contribution to both sides.

At most once

To prove that no resource is ever released twice, we find an *invariant*: a property of states that holds at the start and that every statement preserves.

```
/-- The invariant of every reachable state. -/
structure Inv (s : State) : Prop where
  /-- Owners plus releases never exceed acquisitions. -/
  conserve : ∀ r, held s.slots r + released s.log r ≤ acquired s.log r
  /-- Each resource is acquired at most once. -/
  once : ∀ r, acquired s.log r ≤ 1
  /-- Resources from `next` on have not been acquired yet. -/
  fresh : ∀ r, s.next ≤ r → acquired s.log r = 0
```

The first field is a conservation law: owners plus releases never exceed acquisitions. The second says that each resource is acquired at most once, the third that resources from `next` on have not been acquired yet (which makes `new` safe). A `structure` in `Prop` bundles several proofs: `Inv s` holds when all three fields do.

Here is the preservation proof, one case per statement:

```
theorem step_inv {s s' : State} {st : Stmt} (hs : Inv s)
    (h : step s st = .ok s') : Inv s' := by
  obtain ⟨hcons, honce, hfresh⟩ := hs
  cases st with
  | new x =>
    -- A fresh resource `s.next` gets one owner and one acquisition.
    simp only [step, Except.ok.injEq] at h
    subst h
    -- Before the step, `s.next` had no owner and no release.
    have h0 := hfresh s.next (Nat.le_refl _)
```

```
    have hc0 := hcons s.next
    refine ⟨fun r => ?_, fun r => ?_, fun r hr => ?_⟩
    · have := hcons r
      by_cases hr : s.next = r
      · subst hr; simp; omega
      · simp [hr]; omega
    · have := honce r
      by_cases hr : s.next = r
      · subst hr; simp; omega
      · simp [hr]; omega
    · -- `r` is at least `s.next + 1`: it is neither `s.next` nor acquired.
      have hlt : s.next < r := hr
      simp [Nat.ne_of_lt hlt, hfresh r (Nat.le_of_lt hlt)]
  | move x y =>
    -- The resource changes owner: every count is unchanged.
    simp only [step] at h
    split at h
    · rename_i r slots htake
      simp only [Except.ok.injEq] at h
      subst h
      refine ⟨fun q => ?_, honce, hfresh⟩
      have h1 := take_held htake q
      have h2 := hcons q
      simp only [held_cons, Option.some.injEq]
      omega
    · simp at h
  | drop x =>
    -- One owner fewer, one release more.
    simp only [step] at h
    split at h
    · rename_i r slots htake
      simp only [Except.ok.injEq] at h
      subst h
      refine ⟨fun q => ?_, fun q => ?_, fun q hq => ?_⟩
      · have h1 := take_held htake q
        have h2 := hcons q
        simp only [released_release, acquired_release]
        omega
      · simpa using honce q
      · simpa using hfresh q hq
    · simp at h
  | forget x =>
    -- One owner fewer, and nothing in the log.
    simp only [step] at h
    split at h
    · rename_i r slots htake
      simp only [Except.ok.injEq] at h
      subst h
      refine ⟨fun q => ?_, honce, hfresh⟩
      have h1 := take_held htake q
      have h2 := hcons q
      dsimp only
      omega
    · simp at h
  | read x =>
    simp only [step] at h
    split at h
    · simp only [Except.ok.injEq] at h
      subst h
      exact ⟨hcons, honce, hfresh⟩
    · simp at h
```

Each case starts the same way: `simp only [step] at h` unfolds the evaluator in `h : step s st = .ok s'`, `split at h` separates the success and failure of `take`, and `subst h` replaces `s'` by the state the step built. Failure cases end with `simp at h`, since `.error e = .ok s'` is impossible. Then:

- **`new`**: the fresh resource `s.next` gains one owner and one acquisition. `by_cases hr : s.next = r` separates it from the others. It had not been acquired (`h0`), hence (by `hc0`) had no owners or releases, and its new counts are 1, 0 and 1; the other counts do not change. `simp` applies the counting lemmas and `omega` does the arithmetic. For the third field, `hr` is `s.next < r` once the structure is unfolded, which is all `have hlt` says.
- **`move`**: by `take_held`, the old slots had one more owner of `r` than the remaining ones, and the new slot `(y, some r)` adds it back: nothing changes.
- **`drop`**: one owner fewer (`take_held`), one release more (`released_release`); the sum is unchanged.
- **`forget`**: one owner fewer and no release. The sum *decreases*, and `≤` survives a decrease. This is the only place where `forget` matters, and it already shows that an equality would not survive.
- **`read`**: nothing changes.

Induction over the statements gives the same for whole programs (`exec_inv` in the file), and the theorem follows:

```
/-- **At most once.** For every program, whatever it does, no resource
appears twice as released in the final log. -/
theorem run_at_most_once {prog : List Stmt} {log : List Event}
    (h : run prog = .ok log) (r : Res) : log.count (.release r) ≤ 1 := by
  simp only [run] at h
  split at h
  · rename_i s hexec
    simp only [Except.ok.injEq] at h
    subst h
    have hs := exec_inv inv_init hexec
    have h1 := released_dropAll s.slots s.log r
    have h2 := hs.conserve r
    have h3 := hs.once r
    simp only [released] at h1 h2
    omega
  · simp at h
```

After unfolding `run`, the final log is `dropAll s.slots s.log`, where `s` is the final state. Lemma `released_dropAll` (proved in the file by induction on the slots) says that the end of the scope adds one release per owner. So the final releases are the earlier releases plus the owners, which by the invariant is at most the acquisitions, which is at most 1; `omega` chains the facts. The theorem holds for *every* program, with `forget` or without: a resource can only be released by taking it out of a slot, and taking it clears the slot.

Exactly once, and how `forget` breaks it

For "exactly once" we need the equality version of the conservation law, and we need to exclude `forget`:

```
/-- A state is *balanced* when owners plus releases equal acquisitions:
nothing has been lost. -/
def Balanced (s : State) : Prop :=
  ∀ r, held s.slots r + released s.log r = acquired s.log r

def NoForget : Stmt → Prop
  | .forget _ => False
  | _ => True
```

`NoForget` is a predicate on statements defined by pattern matching: it is the proposition `False` for `forget` and `True` for everything else. The proof that `Balanced` is preserved by every statement other than `forget` (`step_balanced` in the file) is the same case analysis as before with `=` in place of `≤`; in the `forget` case the hypothesis `hf : NoForget (.forget x)` is a proof of `False`, and `absurd` concludes anything from it. Then:

```
/-- **Exactly once.** Without `forget`, every resource that was acquired is
released exactly once by the end of the scope. -/
theorem run_exactly_once {prog : List Stmt} {log : List Event}
    (hf : ∀ st ∈ prog, NoForget st) (h : run prog = .ok log)
    (r : Res) (hr : Event.acquire r ∈ log) : log.count (.release r) = 1 := by
  simp only [run] at h
  split at h
  · rename_i s hexec
    simp only [Except.ok.injEq] at h
    subst h
    have hs := exec_inv inv_init hexec
    have hb := exec_balanced (fun r => by
      simp [State.init, held, released, acquired]) hf hexec
    -- The final log has the same acquisitions, and one release per owner.
    have h1 := released_dropAll s.slots s.log r
    have h2 := acquired_dropAll s.slots s.log r
    have h3 := hb r
    have h4 := hs.once r
    have h5 : 0 < acquired (dropAll s.slots s.log) r := List.count_pos_iff.mpr hr
    simp only [released, acquired] at h1 h2 h3 h4 h5
    omega
  · simp at h
```

At the end of a program without `forget`, owners plus releases equal acquisitions (`hb`); the end of the scope turns every owner into a release (`h1`) and acquires nothing (`h2`), so in the final log releases equal acquisitions. Acquisitions are at most 1 (`h4`) and at least 1 (`h5`: `List.count_pos_iff` turns "`acquire r` is in the log" into "its count is positive"). So releases are exactly 1.

Finally, the hypothesis `hf` cannot be dropped:

```
/-- **`forget` breaks "exactly once".** The statement of `run_exactly_once`
without its hypothesis is false: one program is a counterexample. -/
theorem forget_breaks_exactly_once :
    ¬ ∀ (prog : List Stmt) (log : List Event), run prog = .ok log →
        ∀ r, Event.acquire r ∈ log → log.count (.release r) = 1 := by
  intro claim
  -- the program `let conn = ...; mem::forget(conn);`
  have h := claim [.new "conn", .forget "conn"] [.acquire 0] rfl 0 (by simp)
  -- its log has no `release`: the count is 0, not 1
  simp at h
```

This proves a negation: we assume the general claim and derive a contradiction. We apply it to the program that creates a resource and forgets it, whose log is `[.acquire 0]`; the argument `rfl` proves `run [...] = .ok [.acquire 0]` by running the program. The claim then says that `release 0` occurs exactly once in `[.acquire 0]`; `simp at h` computes the count, finds `0 = 1`, and closes the goal.

Affine in typing: a move checker

The evaluator detects use-after-move at run time, by failing. Rust programs carry no ownership information at run time: the type checker guarantees the failure cannot happen. The last part of the file makes that precise with a static checker that sees only names and flags, never resources:

```
/-- The checker's view of a slot: the name and whether it is initialized. -/
abbrev Flag := Var × Bool

def takeFlag (x : Var) : List Flag → Option (List Flag)
  | [] => none
  | (y, b) :: rest =>
    if y = x then (if b then some ((y, false) :: rest) else none)
    else (takeFlag x rest).map ((y, b) :: ·)

def checkStmt (flags : List Flag) : Stmt → Option (List Flag)
  | .new x => some ((x, true) :: flags)
  | .move x y => (takeFlag x flags).map ((y, true) :: ·)
  | .drop x => takeFlag x flags
  | .forget x => takeFlag x flags
  | .read x => (takeFlag x flags).map (fun _ => flags)

/-- Accept a program if no statement uses a moved (or undeclared) variable. -/
def check (flags : List Flag) : List Stmt → Bool
  | [] => true
  | st :: prog =>
    match checkStmt flags st with
    | some flags' => check flags' prog
    | none => false

/-- info: false -/
#guard_msgs in
#eval check [] [.new "conn", .drop "conn", .read "conn"]
```

`takeFlag` is `take` with a `Bool` instead of an `Option Res`. This is the affine discipline of Chapter 22 in miniature: a variable may be consumed at most once, and not used afterwards. Its soundness theorem:

```
/-- **Well-checked programs do not go wrong**: if the move checker accepts a
program, running it never reports a use of a moved value. -/
theorem check_sound {prog : List Stmt} (h : check [] prog = true) :
    ∃ log, run prog = .ok log := by
  obtain ⟨s, hs⟩ := exec_of_check (s := State.init) h
  exact ⟨dropAll s.slots s.log, by simp [run, hs]⟩
```

The proof (in the file) relates the two views with a function `erase` that keeps only the flags, and shows by induction that whenever the checker can take a flag, the evaluator can take the resource, and the erased states stay equal. Together with the previous theorems, the picture is complete:

- **typing** (`check`) is affine: it allows weakening (a variable may be left alone) and forbids contraction (no use after a move);
- **dynamics** (`run`) is linear without `forget` : every acquired resource is released exactly once, because the end of the scope inserts the releases the program did not perform;
- `forget` is the one operation that is well typed and breaks linearity of the dynamics, while keeping "at most once".

That is the slogan of this chapter, now as theorems.

24.8 Putting it together: a transaction, from Rust to the log

Let us follow the `Transaction` of `ch24_raii.rs` through the whole chapter. `Transaction::begin` acquires a resource (in a real database, a transaction on the server); in the log model, that is `new tx` . The type is not `Copy` and has a destructor, so it will be released exactly once, unless it is forgotten.

There are two ways to consume it. `tx.commit()` takes `self`: in the model, `move tx c`, and the end of `commit`'s scope drops `c`. Inside `commit`, the rows reach the table through `append`, because moving `pending` out of a `Drop` type is forbidden (E0509); then `committed` is set, and the destructor, which still runs at the end of `commit`, stays silent. The other way is to drop it, on any path out of the scope, and the destructor rolls back. Either way the transaction is released exactly once, as `run_exactly_once` says for programs without `forget`.

What about `mem::forget(tx)`? Nothing is committed, the destructor does not run: the resource leaks, as in `forget_breaks_exactly_once`. The design is still *sound*, because the table is modified only inside `commit`: a forgotten transaction loses its pending rows, which is what a rollback would have done. That is the rule of the Leakpocalypse applied. Had we written the rows immediately and relied on the destructor to *remove* them on rollback, a forgotten transaction would leave half-finished data behind.

A linear `Transaction` would go further: the compiler would reject a function that neither commits nor rolls back, and `rollback` could be an explicit method returning a `Result`. Today, `#[must_use]` warns about the most common slip, and the destructor catches the rest at run time.

24.9 Common confusions

"**A value is dropped when its variable goes out of scope.**" Only if the variable still owns it. A value moved into a function is dropped when that function's parameter goes out of scope; a value moved into a collection is dropped with the collection; a value whose variable was moved from is not dropped at that variable's scope end at all. Ownership, not scope, decides.

"**`let _ = x;` drops `x`.**" It does not even move it. The pattern `_` binds nothing, so when the right-hand side is a variable, `let _ = x;` does nothing at all, and `x` lives on. When the right-hand side is a temporary, such as `let _ = make_guard();`, the temporary is dropped at the end of the statement, which is usually not what the author wanted. Use `drop(x)` to drop, and a named variable such as `_guard` to keep.

"**Leaking memory is unsafe.**" It is safe, and sometimes intended (`Box::leak`). It is usually a bug, but like an infinite loop, not like a use-after-free; unsafe code must stay sound even when values are leaked.

"**`Rc` cleans up cycles eventually.**" It never does. Reference counting frees a value when its count reaches zero, and a cycle keeps every count in it above zero. Use `Weak` for back edges.

24.10 Summary

- Rust keeps the typing rule of weakening but gives discarding an effect, the destructor: "affine in typing, linear in dynamics".
- RAII ties resources to values, released on every path out of a scope, including panics. Destructors cannot take arguments or report errors, so good APIs also offer an explicit consuming method.
- Drop order is fixed: locals in reverse declaration order; fields and elements in declaration order, after the outer `Drop::drop`; temporaries at the end of their statement; `let _ =` binds nothing.

Moves transfer the duty to drop, and drop flags handle conditional moves. Assignment drops the old value; `mem::drop` is weakening as a function.

- Drop glue runs `Drop::drop` and then the fields' glue. To keep "at most once", the compiler forbids calling `Drop::drop` (E0040), moving fields out of a `Drop` type (E0509) and `Copy` with `Drop` (E0184). `ManuallyDrop` turns the glue off.
- Rust is affine, not linear: `mem::forget` and `Box::leak` are safe, and `Rc` cycles leak even without them. Since 2015, soundness must not depend on a destructor running; `Vec::drain` and `thread::scope` show how to design for it.
- Linear types would allow mandatory consumption, fallible cleanup and protocol completion. `#[must_use]` is only a lint; Rust 1.99 has no way to declare a type that must be consumed.
- Lean has no destructors; reference counting frees memory deterministically and lets pure code update unshared values in place, invisibly to the logic.
- For a small resource language with a drop log we proved in Lean: no resource is released twice; without `forget`, every resource is released exactly once; `forget` breaks the second property; an affine move checker rules out use-after-move.

24.11 Exercises

1. Predict the output of the following function, then check your answer by adding it to a copy of `ch24_drop_order.rs`:

```
fn puzzle() {
    let a = Noisy("a");
    let v = vec![Noisy("v0"), Noisy("v1")];
    let a = Noisy("a2");
    let _ = Noisy("tmp");
    let b = (Noisy("b0"), a);
    println!("  end of scope");
}
```

   Hint: shadowing does not drop the shadowed value; the first `a` is still alive (and still owned by its now unnameable variable). The second `a` is moved into the tuple. Locals go in reverse order, elements in order.

2. In `ch24_leaks_and_scopes.rs`, change the cycle so that `b` points back to `a` with a `Weak` instead of an `Rc`. Predict the counts printed by `counts` and the destructor messages, then run it. *Hint:* `a` then has one strong and one weak reference; when the locals go, `a`'s strong count reaches zero first, and its `children` vector releases `b`.

3. A `Pool` hands out a buffer with `fn borrow_buffer(&self) -> Guard<'_>`, and the guard's destructor marks the buffer free again. Unsafe code elsewhere reuses the buffer as soon as it is marked free. Is the API sound if a guard is forgotten? Would it be if the guard's destructor were what *stopped* other code from writing into the buffer? Redesign the second version in the style of `thread::scope`. *Hint:* a forgotten guard that keeps the buffer marked busy is only a leak. A forgotten guard that never revokes someone's access is the `thread::scoped` bug. A closure-based `with_buffer(&self, f: impl FnOnce(&mut [u8]))` does the cleanup itself after `f` returns.

4. In `DropLog.lean`, add a statement `swap (x y : Var)` that exchanges the resources owned by two variables (failing if either is moved out). Extend `step`, `checkStmt` and the proofs. Does `step_inv` need a new idea? *Hint:* implement it with two `take`s and two new slots; then each count of owners is unchanged, as for `move`, and the `move` case of the proofs is a good template.
5. Prove in `DropLog.lean` that the checker rejects `[.new "a", .drop "a", .drop "a"]` and that running it fails, both by `rfl`. Then suppose `move` forgot to clear the drop flag of `x`. Find a program whose log releases a resource twice, and say which field of `Inv` the `move` case of `step_inv` could no longer preserve. *Hint:* two statements suffice, and the end of the scope does the rest. Count the owners of `r0` after the move.
6. In `InPlace.lean`, write `reverseInPlace (xs : Array Nat) : Array Nat` that swaps elements `i` and `n - 1 - i` for `i < n / 2`, with `dbgTraceIfShared` before each swap. Check with `#guard_msgs` that reversing `Array.range n` (built inside a function of `n`) prints no `shared RC` message, then make a small change that makes it print one. *Hint:* `Array.swapIfInBounds xs i j` swaps two positions. To force a copy, keep the original array alive, for example by returning it in a pair as `bumpAndKeep` does.